

TCP拥塞控制算法



1.Tahoe

- Slow Start(慢启动)

(`ssthresh`:slow start thresh,慢启动门限值)

当 `cwnd` 的值小于 `ssthresh` 时, TCP则处于 `slow start` 阶段, 每收到一个ACK, `cwnd` 的值就会加1。

仔细分析, 其实慢启动并不慢, 经过一个RTT的时间, `cwnd` 的值就会变成原来的两倍, 实为指数增长。

- Congestion Avoidance (拥塞避免)

当 `cwnd` 的值超过 `ssthresh` 时, 就会进入**Congestion Avoidance**阶段, 在该阶段下, `cwnd` 以线性方式增长, 大约每经过一个RTT, `cwnd` 的值就会加1。

- Fast Retransmit (快速重传)

按照拥塞避免算法中 `cwnd` 的增长趋势, 迟早会造成拥塞 (一般通过是否丢包来判断是否发生了拥塞)。

如果中网络中发生了丢包, 通过等待一个RTO时间后再进行重传, 是非常耗时的, 因为RTO通常设置得会比较大 (避免伪重传: 不必要的重传)。

快重传的思想是: 只要发送方收到了三个重复的ACK (如果了解三个重复ACK如何产生, 请温故一下滑动窗口协议), 就会立马重传, 而不用等到RTO到达 (如果没有3个重复的ACK而包丢失了, 就只能超时重传);

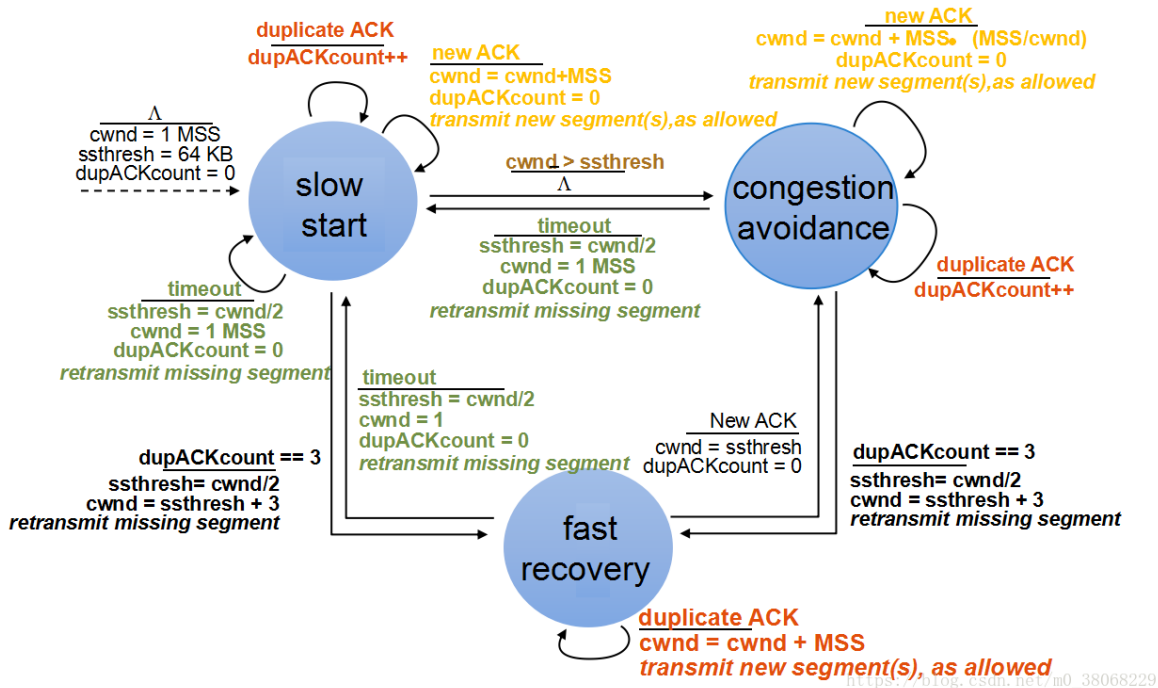
并且将`ssthresh`的值设置为当前 `cwnd` 的一半, 而 `cwnd` 减为1, 重回**slow start**阶段。

2.Reno

Reno多了一个快速恢复算法。

当收到三个重复的ACK或是超过了RTO时间且尚未收到某个数据包的ACK, Reno就会认为丢包了, 并认定网络中发生了拥塞。

Reno会把当前的 `ssthresh` 的值设置为当前 `cwnd` 的一半, 但是并不会回到 `slow start` 阶段, 而是将 `cwnd` 设置为 (更新后的) `ssthresh+3MSS`, 之后 `cwnd` 呈线性增长。



3.NewReno

当丢失一个分组时可以采用Reno算法，但是当丢失多个分组时如果还使用Reno算法则不适用。

Reno快速恢复算法中发送方收到一个新的ACK就退出快速恢复状态，New Reno算法中只有当所有报文都被应答后才退出快速恢复状态。

NewReno TCP添加了恢复应答判断功能，以增强TCP终端通过ACK报文信息分析报文传输状况的能力。

使TCP终端可以把一次拥塞丢失多个报文的情形与多次拥塞的情形区分开来，进而在每一次拥塞发生后拥塞窗口仅减半一次，从而提高了TCP的顽健性和吞吐量。

两个概念：部分应答（PACK）、恢复应答（RACK）

记TCP发送端恢复阶段中接收到的ACK报文（非冗余ACK）为ACK_x，记在接收到ACK_x时TCP终端已发出的序列号（SN）最大的报文是PKTy，如果ACK_x不是PKTy的应答报文，则称报文ACK_x为部分应答（Partial ACK，简称PACK）；若ACK_x恰好是PKTy的应答报文则称报文ACK_x为恢复应答（Recovery ACK，简称RACK）。

举例来理解：如果4、5、6号包丢了，现在只重传4，只收到了4的ACK，后面的5、6没有确认，这就是部分应答Partial ACK。如果收到了6的ACK，则是恢复应答Recovery ACK。

TCP发送端接收到恢复应答表明：经过重传，TCP终端发送的所有报文都已经被接收端正确接收，网络已经从拥塞中恢复。

重新定义快速恢复阶段：

1. 进入恢复阶段后，发送端重传被认定为丢失的报文，设置慢启动阈值（ $ssthresh$ ）和拥塞窗口大小（ $cwnd$ ）。 $ssthresh = cwnd/2$, $cwnd = ssthresh + 3MSS$ 。
2. 收到一个重复ACK, $cwnd = cwnd + MSS$ 。
3. 当收到PACK（部分应答）时，发送端重传PACK所确认报文的下一个报文，如果拥塞窗口允许，继续发送新的数据包。
4. 当收到RACK（确认应答）时，发送端认为拥塞中所有被丢弃的报文都已经被重传，拥塞结束，设置 $cwnd = ssthresh$ 并退出快速恢复状态。

4.Sack

虽然NewReno可以解决大量数据包遗失的问题，但是NewReno在每个RTT时间只能一个数据包遗失的错误。为了更有效地处理大量数据包遗失的问题，另一个解决方法就是让传送端知道哪些已经被接收端收到，但用此方法必须同时修改传送端和接收端的传送机制。

TCP SACK在TCP Reno基础上增加了：

1. 选择确认 (Selective Acknowledgements, SACK)
2. 选择重传 (Selective Retransmission)

关于SACK详见：[SACKcsdn](#)

5.CUBIC

CUBIC和Reno在CA阶段不同，假定拥塞窗口可达到的最大容量为K，在CA阶段，从 t_0 开始，到K的这段时间， t 和W采取一个立方的函数关系，在 t 增加少量时，cwnd的大小增加很快，在接近K时，cwnd的大小增加变慢，试探链路容量。

6.Vegas

基于时延的TCP

7.BBR

8.Others
